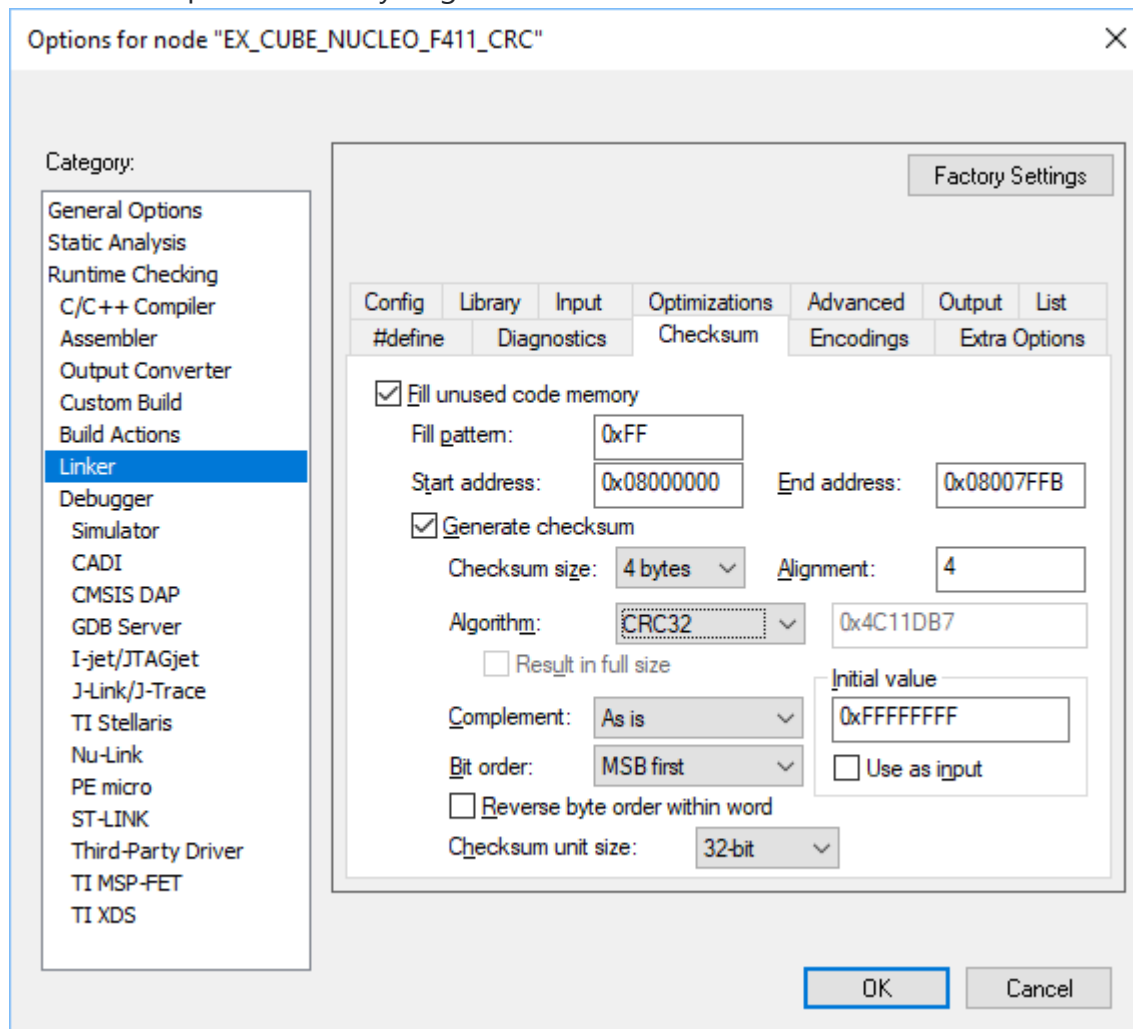


	ethanhuanginst Fix Layout Issue of README Files ...	on 21 May 2019	History
..			
	Drivers	3 years ago	
	Inc	3 years ago	
	Src	3 years ago	
	Startup	3 years ago	
	.cproject	3 years ago	
	.mxproject	3 years ago	
	.project	3 years ago	
	06_F746-DISCO-CRC.ioc	3 years ago	
	README.md	3 years ago	
	STM32F746NGHX_FLASH.Id	3 years ago	
	STM32F746NGHX_RAM.Id	3 years ago	
	checksum.bat	3 years ago	

Hands-on: CRC Checksum Generation

Introduction

IAR EWARM provides a way to generate checksum as shown below:

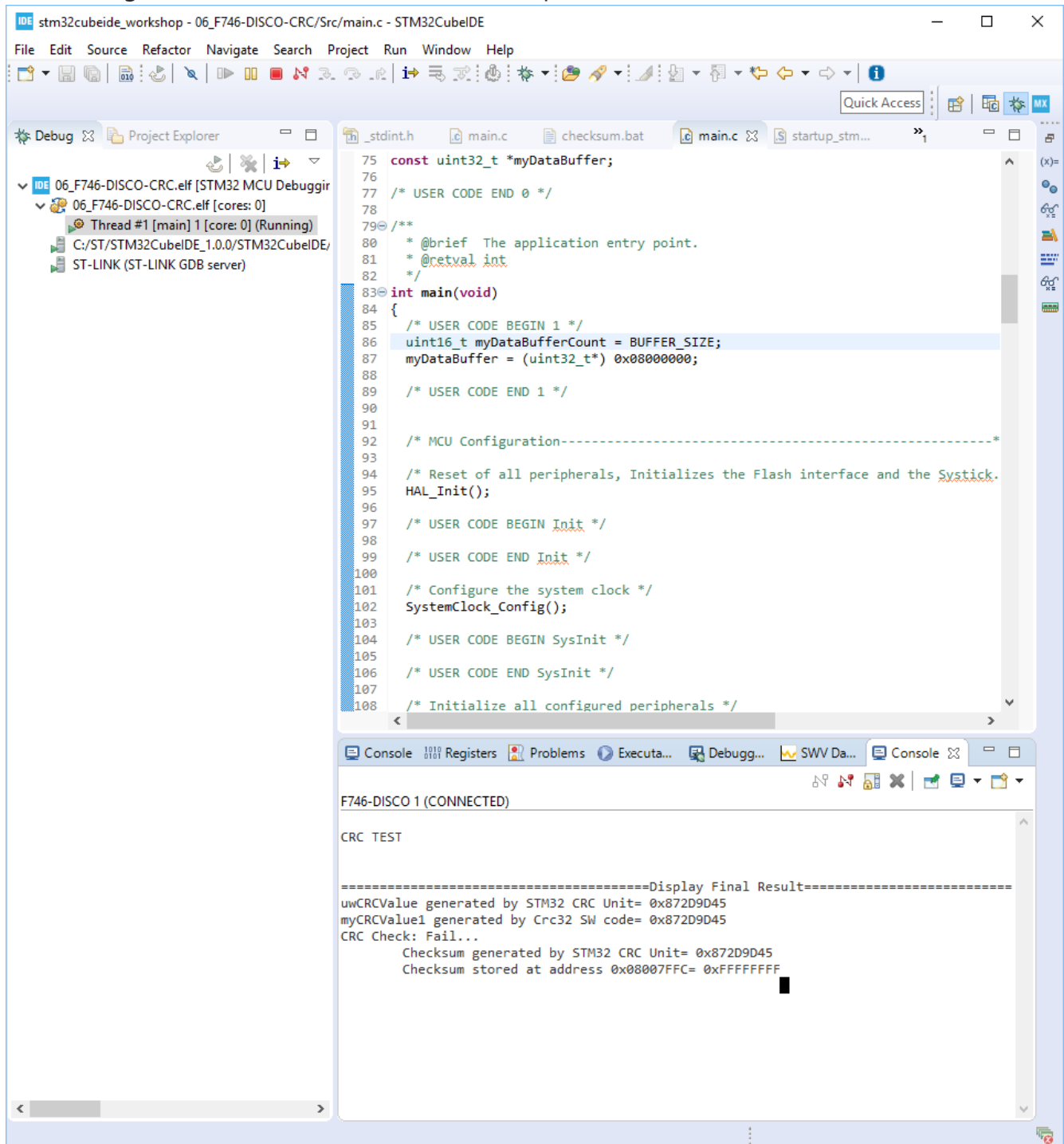


How can this be done by using STM32CubeIDE? This hands-on provides an option for you.

Steps

1. Import the reference project.
2. Build Project

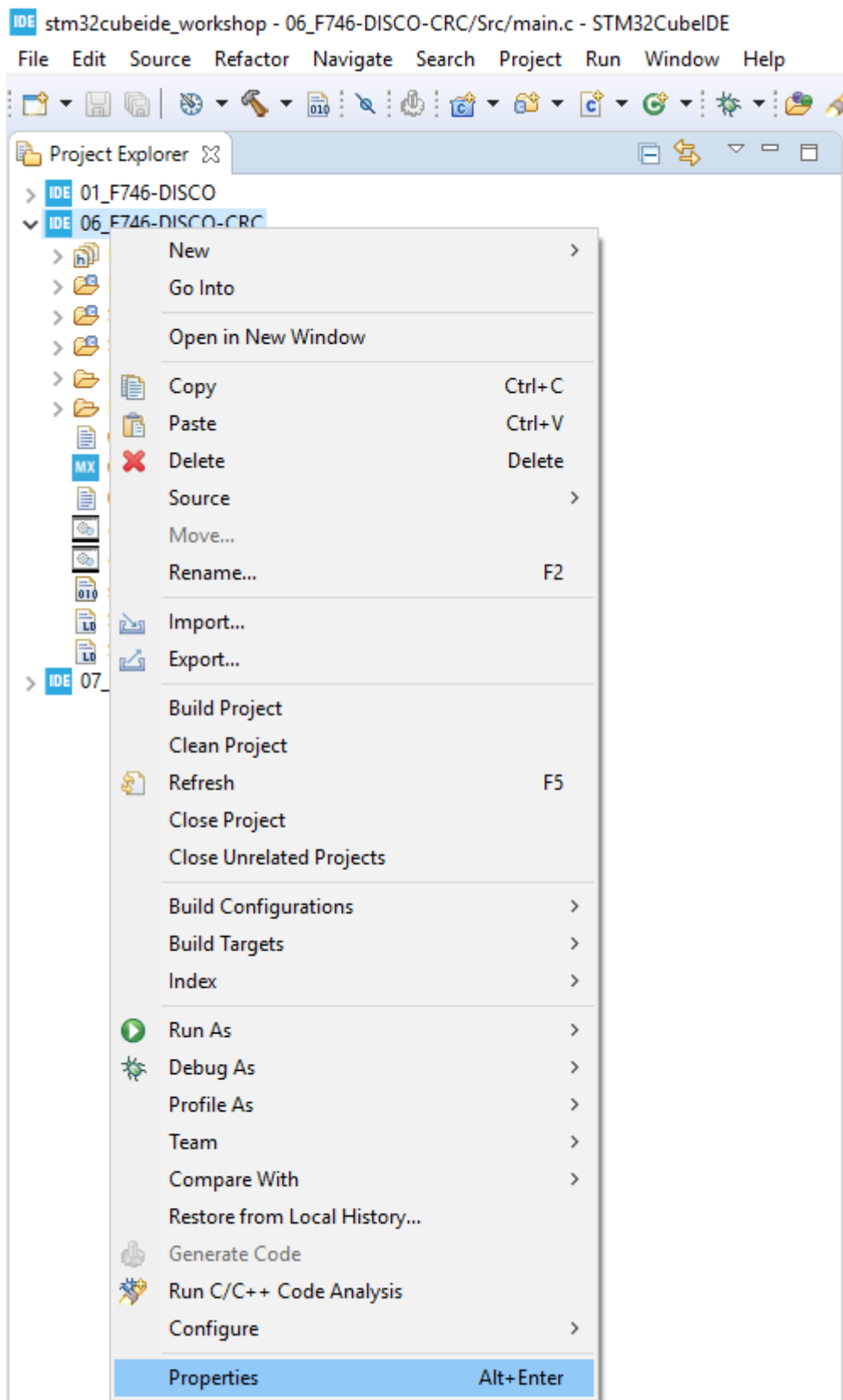
3. Start Debug session and check result from serial port console as shown below:



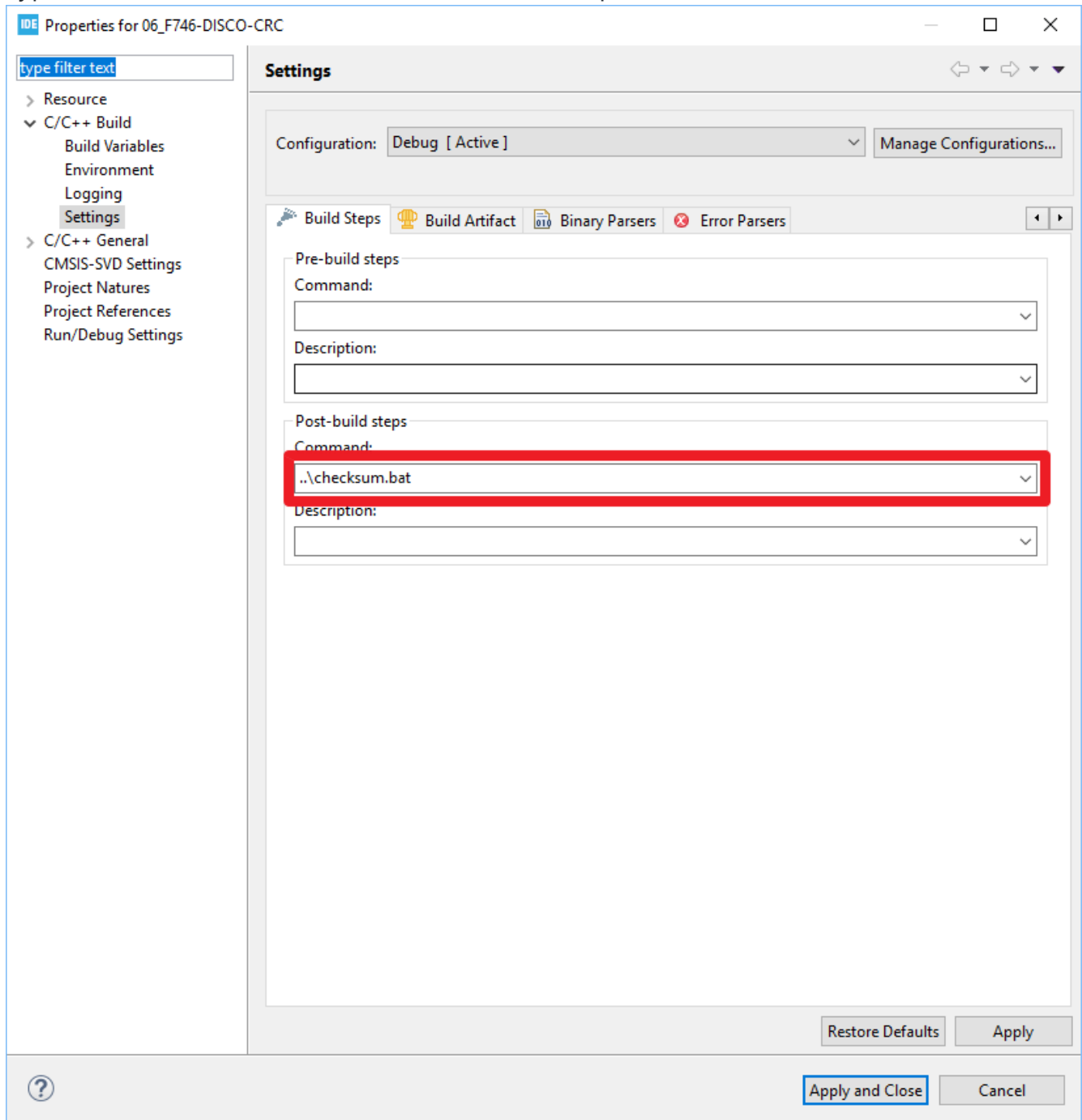
- CRC generated by CRC unit in STM32F746 is 0x872D0D45.
- CRC generated by CRC code running over STM32F746 is also 0x872D0D45.
- Checksum stored in the last word of the first sector is 0xFFFFFFFF which is not aligned with that generated by CRC unit and therefore CRC check fails. This is expected since we have not put any checksum into the HEX/BIN file yet.

4. [Srec_cat](#) could be used to generate CRC checksum and put it into HEX file. To simplify the process, please put srec_cat.exe into the root of project folder.

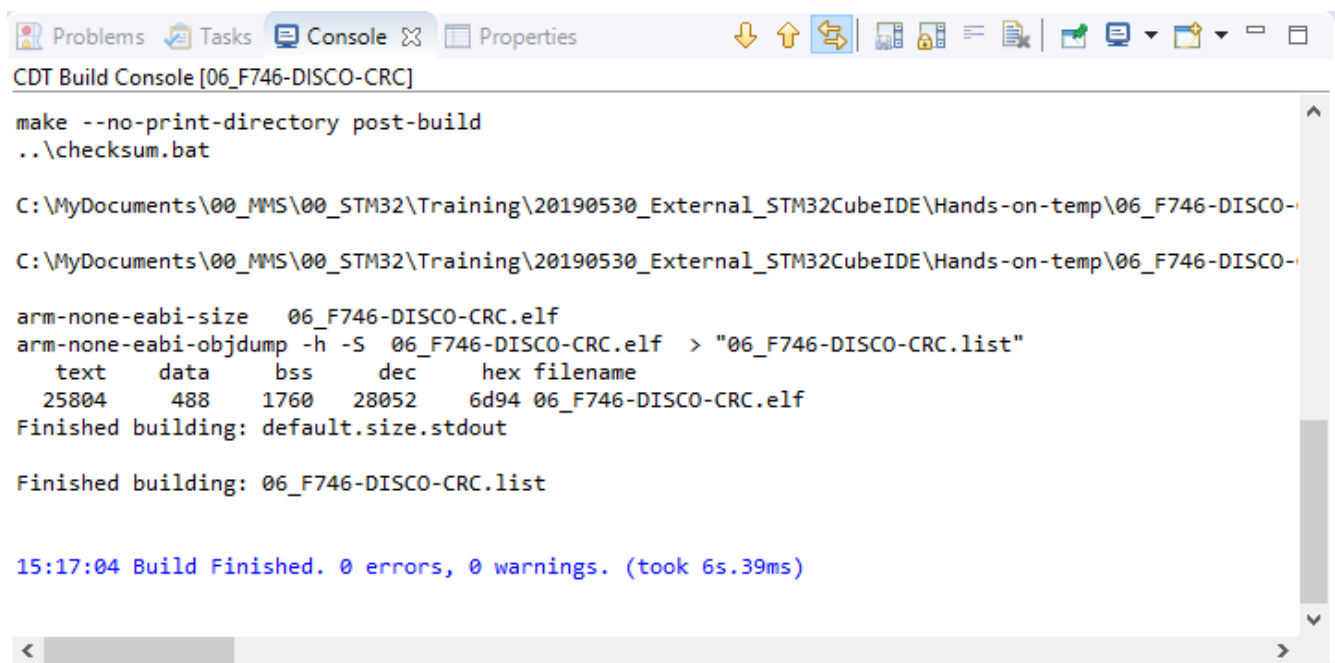
5. Right click on project icon and then select "Properties":



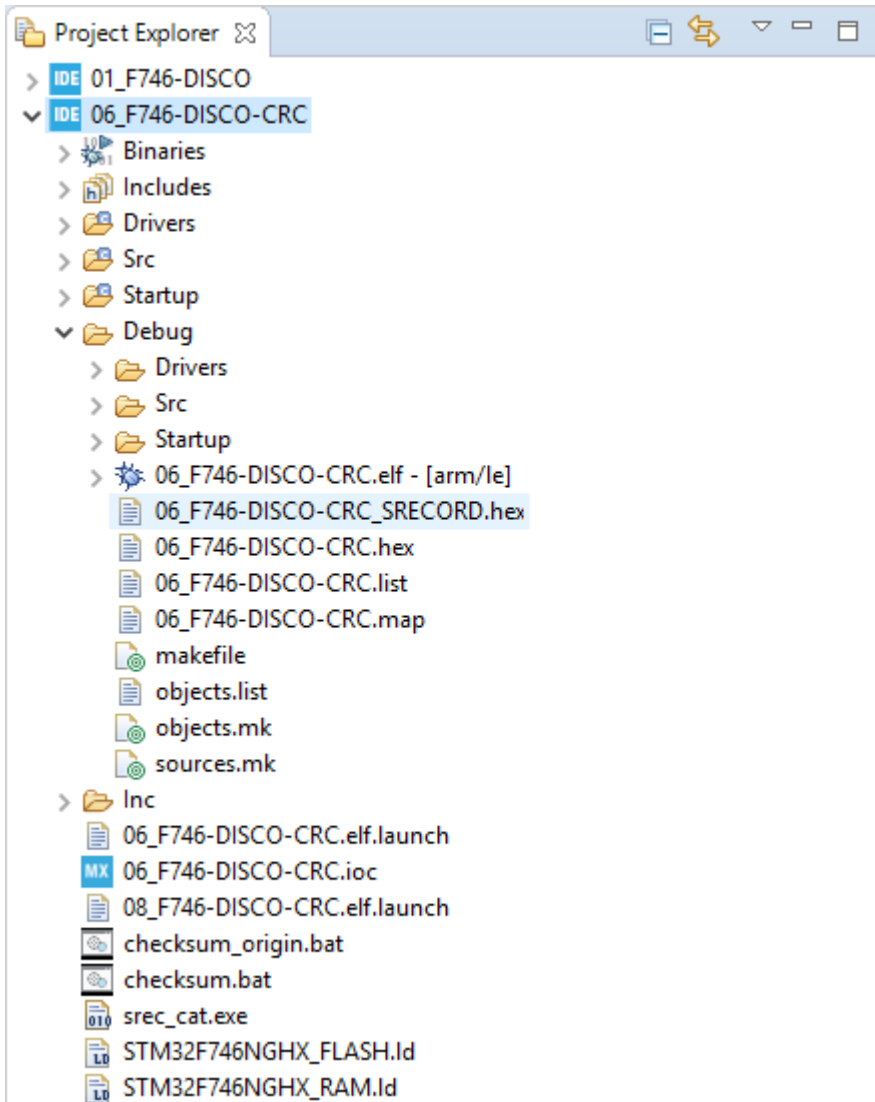
6. Type "..\checksum.bat" in Command of Post-build steps:



7. "Clean Project" and then "Build Project" again and we can see checksum.bat executes correctly without error:



8. Check files under "Debug" folder and **06_F746-DISCO-CRC_SRECORD.hex** generated by checksum.bat is found:



9. Check the last word of 06_F746-DISCO-CRC_SRECORD.hex by STM32CubeProgrammer. CRC checksum generated by srec_cat and put in the last word of **06_F746-DISCO-CRC_SRECORD.hex** (0x872D9D45) is exactly the same as that by CRC unit of STM32F746 (0x872D9D45).

The screenshot displays the STM32CubeProgrammer interface. The main window is titled "Memory & File edition" and shows a memory editor for the file "06_F746-DISCO-CRC_SRECORD.hex". The address range is set from 0x8000000 to 0x8000000 with a 32-bit data width. The memory content is displayed in a table with columns for Address, 0, 4, 8, C, and ASCII. The last word of the memory, at address 0x08007FF0, is highlighted with a red box and contains the value 872D9D45. The right sidebar shows the ST-LINK configuration, including Serial number, Port, Frequency, Mode, Access port, Reset mode, Shared, External loader, Target voltage, and Firmware version.

Address	0	4	8	C	ASCII
0x08007EF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F00	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F10	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F20	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F30	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F40	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F50	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F60	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F70	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F80	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007F90	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FA0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FB0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FC0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FD0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FE0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF	yyyyyyyyyyyyyy
0x08007FF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	872D9D45	yyyyyyyyyyE.-.

10. After running the debug session and checking the result in serial port console, we still get CRC check failure. And memory address 0x08007FFC which is supposed to have CRC checksum 0x872D9D45 still contains 0xFFFFFFFF:

The screenshot displays the STM32CubeIDE interface during a debug session. The main.c file is open, showing the main function. The assembly view on the right shows the initialization of the CRC unit. The memory monitor on the right shows the address 0x08007FFC containing 0xFFFFFFFF instead of the expected 0x872D9D45. The console at the bottom shows the final result of the CRC check, indicating a failure.

```
75 const uint32_t *myDataBuffer;
76
77 /* USER CODE END 0 */
78
79 /**
80  * @brief The application entry
81  * @retval int
82  */
83 int main(void)
84 {
85     /* USER CODE BEGIN 1 */
86     uint16_t myDataBufferCount = BUFFER_SIZE;
87     myDataBuffer = (uint32_t*) 0x08000000;
88     /* USER CODE END 1 */
89
90     /* MCU Configuration-----*/
91
92     /* Reset of all peripherals, Initializes the Flash interface and the Systick timer. */
93     HAL_Init();
94
95     /* USER CODE BEGIN Init */
96
97     /* USER CODE END Init */
98
99     /* Configure the system clock */
100     SystemClock_Config();
101
102     /* USER CODE BEGIN SysInit */
103
104     /* USER CODE END SysInit */
105
106     /* Initialize all configured peripherals */
107     MX_GPIO_Init();
108     MX_USART1_UART_Init();
109     MX_CRC_Init();
110     /* USER CODE BEGIN 2 */
111     printf("\n\rCRC TEST\n\r");
112
113     /*###-1- Compute the CRC by STM32 */
114     uwCRCValue = HAL_CRC_Calculate(&hCRC, myDataBuffer, myDataBufferCount);
115
116     /* USER CODE END 2 */
117 }
```

Assembly view (08003278):

```
add r7, sp, #0
uint16_t myDataBufferCount = BUFFER_SIZE;
movw r3, #8191 ; 0x1fff
strh r3, [r7, #6]
myDataBuffer = (uint32_t*) 0x08000000;
ldr r3, [pc, #172] ; (0x8003330 <main+188>)
mov.w r2, #134217728 ; 0x8000000
str r2, [r3, #0]
HAL_Init();
bl 0x8000ee8 <HAL_Init>
SystemClock_Config();
bl 0x8003364 <SystemClock_Config>
MX_GPIO_Init();
bl 0x80034fc <MX_GPIO_Init>
MX_USART1_UART_Init();
bl 0x800349c <MX_USART1_UART_Init>
MX_CRC_Init();
bl 0x8003458 <MX_CRC_Init>
```

Memory monitor (0x08007FF0):

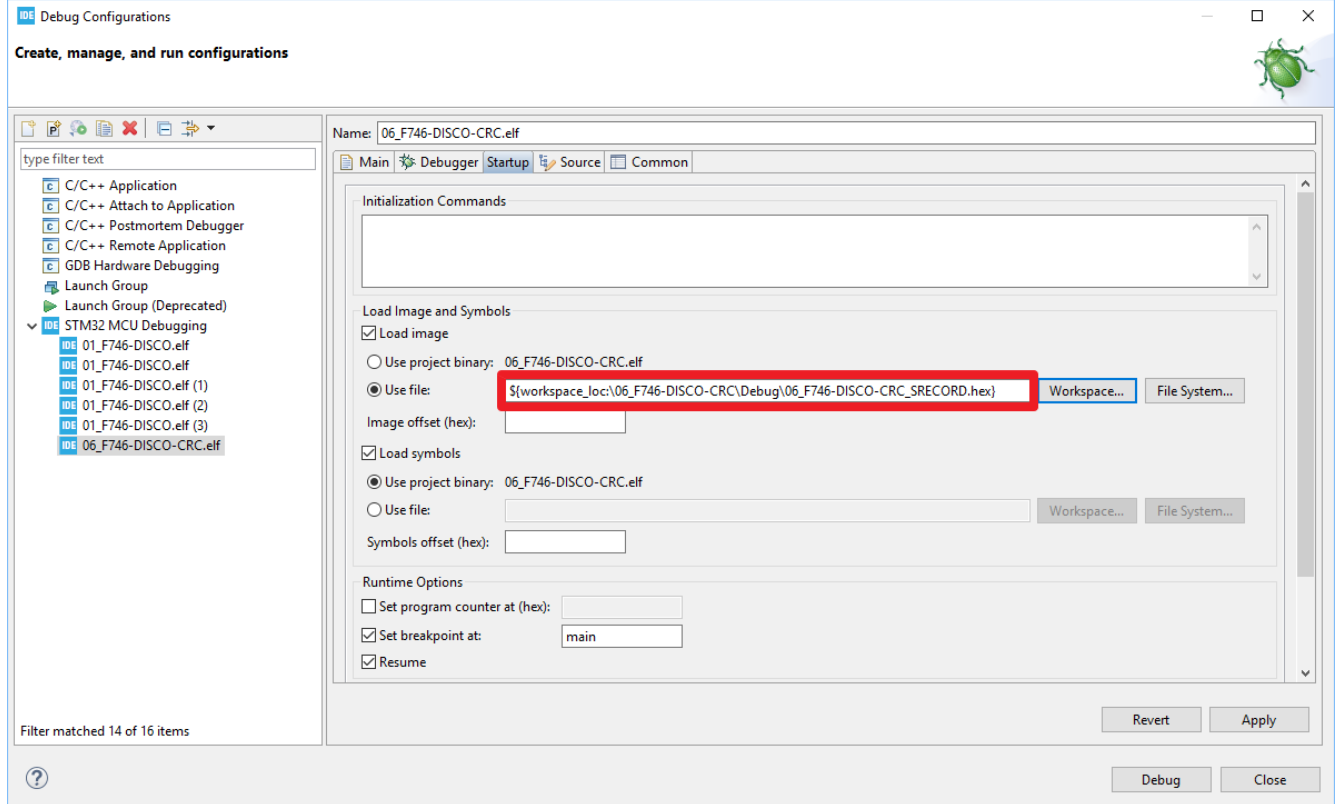
Address	0 - 3	4 - 7	8 - B	C - F
08007F90	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FA0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FB0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FC0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FD0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FE0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08008000	786000BF	0C37BD46	5DF8047B	704780B4
08008010	83B000AF	786000BF	0C37BD46	5DF8047B
08008020	704780B4	83B000AF	786000BF	0C37BD46
08008030	5DF8047B	704780B4	83B000AF	786000BF
08008040	0C37BD46	5DF8047B	704780B4	85B000AF
08008050	78603960	0023FB60	786893F8	3C30012B

Console output:

```
F746-DISCO 1 (CONNECTED)

=====Display Final Result=====
=====
uwCRCValue generated by STM32 CRC Unit= 0x872D9D45
myCRCValue1 generated by Crc32 SW code= 0x872D9D45
CRC Check: Fail...
Checksum generated by STM32 CRC Unit= 0x872D9D45
Checksum stored at address 0x08007FFC= 0xFFFFFFFF
```

11. The reason of this failure is due to the fact that 06_F746-DISCO-CRC_SRECORD.hex (containing CRC checksum) is not programmed when running debug session. We need to assign 06_F746-DISCO-CRC_SRECORD.hex** (containing CRC checksum) to be programmed instead of original 06_F746-DISCO-CRC.elf (w/o CRC checksum). Please open "Debug Configuration" and make the following change:



12. Run debug session again and finally get correct expected result:

stm32cubeide_workshop - 06_F746-DISCO-CRC/Src/main.c - STM32CubeIDE

File Edit Source Refactor Navigate Search Project Run Window Help

Quick Access

main.c startup_stm...

```
75 const uint32_t *myDataBuffer;
76
77 /* USER CODE END 0 */
78
79 /**
80  * @brief The application entry
81  * @retval int
82  */
83 int main(void)
84 {
85     /* USER CODE BEGIN 1 */
86     uint16_t myDataBufferCount = BUF
87     myDataBuffer = (uint32_t*) 0x080
88
89     /* USER CODE END 1 */
90
91     /* MCU Configuration-----
92
93     /* Reset of all peripherals, In
94     HAL_Init();
95
96     /* USER CODE BEGIN Init */
97
98     /* USER CODE END Init */
99
100     /* Configure the system clock *,
101     SystemClock_Config();
102
103     /* USER CODE BEGIN SysInit */
104
105     /* USER CODE END SysInit */
106
107     /* Initialize all configured per
108     MX_GPIO_Init();
109     MX_USART1_UART_Init();
110     MX_CRC_Init();
111     /* USER CODE BEGIN 2 */
112     printf("\n\rCRC TEST\n\r");
113
114     /*##-1- Compute the CRC by STM3
115     //uwCRCValue = HAL_CRC_Calculat
116
```

0800327a: movw r3, #8191 ; 0x1fff
0800327e: strh r3, [r7, #6]
87 myDataBuffer = (uint32_t*) 0x08000000;
08003280: ldr r3, [pc, #172] ; (0x08003330 <main+188>)
08003282: mov.w r2, #134217728 ; 0x80000000
08003286: str r2, [r3, #0]
95 HAL_Init();
08003288: bl 0x8000ee8 <HAL_Init>
102 SystemClock_Config();
0800328c: bl 0x8003364 <SystemClock_Config>
109 MX_GPIO_Init();
08003290: bl 0x80034fc <MX_GPIO_Init>
110 MX_USART1_UART_Init();
08003294: bl 0x800349c <MX_USART1_UART_Init>
111 MX_CRC_Init();
08003298: bl 0x8003458 <MX_CRC_Init>
113 printf("\n\rCRC TEST\n\r");
0800329c: ldr r0, [pc, #148] ; (0x08003334 <main+192>)

Memory

Monitors

0x08007FF0: 0x8007FF0 <Hex> New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
08007F90	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FA0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FB0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FC0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FD0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FE0	FFFFFFFF	FFFFFFFF	FFFFFFFF	FFFFFFFF
08007FF0	FFFFFFFF	FFFFFFFF	FFFFFFFF	459D2D87
08008000	786000BF	0C37BD46	5DF8047B	704780B4
08008010	83B000AF	786000BF	0C37BD46	5DF8047B
08008020	704780B4	83B000AF	786000BF	0C37BD46
08008030	5DF8047B	704780B4	83B000AF	786000BF
08008040	0C37BD46	5DF8047B	704780B4	85B000AF
08008050	78603960	0023FB60	7B6893F8	3C30012B

Console

F746-DISCO 1 (CONNECTED)

CRC TEST

=====Display Final Result=====

uwCRCValue generated by STM32 CRC Unit= 0x872D9D45
myCRCValue1 generated by Crc32 SW code= 0x872D9D45
CRC Check: OK...

Checksum generated by STM32 CRC Unit= 0x872D9D45
Checksum stored at address 0x08007FFC= 0x872D9D45

Writable Smart Insert 86:1